

Execution contract and SRAM

Tiles: **128 query rows × 256 keys**; head dimension: **128**. This PDF is a readable companion to the editable LaTeX source.

Scope. Q/K/V projections and RoPE are already complete. One call processes one batch item, one shared K/V head, and its four query heads. The control CPU repeats the call for 16 batch items and eight K/V heads, then signals kernel completion. Only DMA accesses DRAM.

Explicit numerical assumptions. Arrays take signed INT8 operands and return exact INT16 results when representable. The vector CPU supports FP32 and INT32 arithmetic. Decode the supplied 16-bit activation format using caller-provided metadata before requantizing. These semantics must be confirmed; bit widths alone do not establish them.

Quantization. For real decoded values X , choose $sX = \max(\text{abs}(X))/127$, or $sX = 1$ for an all-zero tensor. Set $X8 = \text{INT8}(\text{clip}(\text{round}(X/sX), -127, 127))$, with zero point zero. K/V each use one scalar scale per shared head; Q uses one per query tile. The caller supplies a positive INT16 output scale per query head.

CPU operation	Meaning and communication cost
DMA_READ / DMA_WRITE	Wait for DMA acceptance, configure a contiguous copy, and return a completion event. Charge 200 cycles per copy.
VSYNC(routine, ...)	Launch a vector routine and wait for completion. Charge 300 cycles per launch.
SA_DOT(array, ...)	Launch one physical SRAM-to-SRAM array microtile. Charge 100 cycles per launch.
WAIT / WAIT_ALL	Wait for operations and their memory writes to finish. Events do not assume an undocumented hardware command queue.

SRAM reservation at $N = 2048$, for one resident K/V head:

Buffers	Size
Original K and V, 16-bit	1 MiB
INT8 K and packed transpose of INT8 V	0.5 MiB
Two signed digit planes for each K/V operand	1 MiB
Query/output ping-pong buffers; score and probability tiles; INT32 products; FP32 state; packing; raw array results; metadata	2 MiB
Total reservation	4.5 MiB < 16 MiB

The workspace contains a 256 KiB raw-partial buffer, sufficient for either SAFE_DOT call. For other sequence lengths, pad K/V to a multiple of 256 and recompute capacity. Head-major DRAM layout is assumed.

1. Control-CPU program

One batch item and one K/V head. Q[0..3] are the associated query heads; all array and vector operands below are in SRAM.

```

1  N = sequence_length; D = 128; BR = 128; BC = 256
2  TR = ceil(N / BR)
3  eK = DMA_READ(K_dram, K16, 2*N*D)
4  eV = DMA_READ(V_dram, V16, 2*N*D)
5  WAIT_ALL(eK, eV)
6  Kdig, Vdig, sK, sV = VSYNC(PREPARE_KV, K16, V16)
7  # Decode, quantize, zero-pad, and cache packed K/V digits.
8
9  for h in range(4):
10     written[0] = written[1] = COMPLETED
11     ready[0] = DMA_READ(Q[h].tile(0), Qin[0],
12                          2*min(BR, N)*D)
13
14     for i in range(TR):
15         slot = i % 2
16         row0 = i*BR
17         nr = min(BR, N-row0)
18         WAIT_ALL(ready[slot], written[slot])
19
20         if i+1 < TR:
21             next_nr = min(BR, N-(i+1)*BR)
22             ready[1-slot] = DMA_READ(Q[h].tile(i+1),
23                                      Qin[1-slot], 2*next_nr*D)
24
25         Q8, sQ, U, m, ell = VSYNC(INIT_QUERY, Qin[slot], nr)
26         # Quantize valid rows, zero-pad Q8 to 128 rows.
27         # Initialize FP32 U=0, m=-infinity, ell=0.
28         J = ceil(min((i+1)*BR, N) / BC)
29
30         for j in range(J):
31             key0 = j*BC
32             S32 = SAFE_DOT(Q8, Kdig[j], n=BC, R=D)
33             P8, alpha, m, ell = VSYNC(SOFTMAX_PACK,
34                                       S32, sQ, sK, m, ell, row0, key0, nr, N)
35             C32 = SAFE_DOT(P8, Vdig[j], n=D, R=BC)
36             U = VSYNC(MERGE, U, alpha, C32, sV)
37
38         O16[slot] = VSYNC(FINISH, U, ell, nr, output_scale[h])
39         written[slot] = DMA_WRITE(O16[slot],
40                                   O[h].tile(i), 2*nr*D)
41
42     WAIT_ALL(written[0], written[1])
43
44 return outputs_for_four_query_heads

```

Buffer ownership. Never reuse an output slot until its DMA write finishes. Query prefetch can overlap current computation. Each DMA submission waits for engine acceptance; simultaneous DMA read/write service is not assumed.

Orientation. SAFE_DOT(L, Wdig) computes $L \times \text{transpose}(W)$. The score call has $W = K8[j]$. The value call has $W = \text{transpose}(V8[j])$, so its mathematical result is $P8 \times V8[j]$.

Indices. This PDF uses zero-based tile indices. The LaTeX main loop uses one-based tile indices with equivalent bounds. Normalize the current state after J visited blocks, not a state for unvisited future blocks.

2. Vector-CPU routines

The control CPU launches these routines with VSYNC and waits before starting dependent array or DMA operations.

```

1  SOFTMAX_PACK(S32, sQ, sK, m, ell, row0, key0, nr, N):
2      S = FP32(S32) * (sQ*sK / sqrt(128))
3      for r in range(128):
4          for c in range(256):
5              if r >= nr or key0+c >= N or key0+c > row0+r:
6                  S[r,c] = -infinity
7
8          new_m = max(m[r], max(S[r,:]))
9          if new_m == -infinity:
10             alpha[r] = 0
11             P[r,:] = 0
12          else:
13             alpha[r] = 0 if m[r] == -infinity \
14                 else exp(m[r] - new_m)
15             P[r,:] = exp(S[r,:] - new_m)
16
17             ell[r] = alpha[r]*ell[r] + sum(P[r,:])
18             m[r] = new_m
19             P8[r,:] = INT8(clip(round(127*P[r,:]), 0, 127))
20     return P8, alpha, m, ell
21
22  MERGE(U, alpha, C32, sV):
23     for r in range(128):
24         U[r,:] = alpha[r]*U[r,:] + (sV/127)*FP32(C32[r,:])
25     return U
26
27  FINISH(U, ell, nr, output_scale):
28     for r in range(nr):
29         Y = U[r,:]/ell[r] if ell[r] > 0 else zeros(128)
30         O16[r,:] = INT16(clip(round(Y/output_scale),
31                             -32768, 32767))
32     return O16

```

Why multiply by alpha? When a larger maximum appears, earlier contributions must shrink. The factor $\exp(\text{old maximum} - \text{new maximum})$ lies between zero and one. The old output is multiplied by this factor; there is no inverse. At the first valid block, alpha is zero.

Why divide the new contribution by 127? P8 stores $\text{round}(127 \times P)$. Reconstructing the weighted values therefore needs both scales: $sV \times (1/127)$. The final division by ell happens only after all visible K/V blocks have been visited.

Accuracy. The denominator uses FP32 exponential weights while the numerator uses their INT8 approximation. Small weights can round to zero. This mixed-precision variant needs accuracy checks on representative inputs. FP32 reads/writes do not themselves cause rounding; arithmetic updates and explicit quantization do.

3. Overflow-safe array products

SAFE_DOT returns an exact INT32 dot product of its INT8 inputs. It uses only representable INT16 partial results from the physical arrays.

Split each operand on the vector CPU. After widening x before $\text{abs}()$, let $x_0 = \text{sign}(x) \times (\text{abs}(x) \bmod 16)$ and $x_1 = \text{sign}(x) \times \text{floor}(\text{abs}(x)/16)$. Then $x = x_0 + 16 \times x_1$. Digits occupy INT8 slots. With at most 128 terms per partial, **$\text{abs}(\text{partial}) \leq 128 \times 15 \times 15 = 28,800$** , which fits signed INT16.

```

1  SAFE_DOT(L8, Wdig, n, R):
2      # L8: [128,R]; logical W: [n,R]; result: [128,n]
3      # n and R are each either 128 or 256.
4      Ldig = VSYNC(SPLIT_AND_PACK_LEFT, L8)
5      events = []
6      # Every D below has its own SRAM destination.
7      for t in range(0, R, 128):
8          rt = min(128, R-t)
9          for a in range(2):
10             for b in range(2):
11                 for col in range(0, n, 16):
12                     for u in range(3):
13                         row = 32*u
14                         WAIT_ACCEPT(SA32)
15                         lp, wp, dp = PACKED_POINTERS(
16                             Ldig, Wdig, D, a, b, row, col, t, 32)
17                         events += SA_DOT(SA32, lp, wp, dp, rt)
18                         if u < 2:
19                             row = 96 + 16*u
20                             WAIT_ACCEPT(SA16)
21                             lp, wp, dp = PACKED_POINTERS(
22                                 Ldig, Wdig, D, a, b, row, col, t, 16)
23                             events += SA_DOT(SA16, lp, wp, dp, rt)
24             WAIT_ALL(events)
25             C32 = VSYNC(RECONSTRUCT, D)
26             # Release D only after reconstruction finishes.
27             return C32
28
29 VECTOR_CPU_RECONSTRUCT(D):
30     C32 = zeros([128,n], INT32)
31     for each stored partial D[t,a,b,row,col]:
32         C32[its output tile] += INT32(D[t,a,b,row,col])*16**(a+b)
33     return C32

```

Packing is explicit. PACKED_POINTERS only calculates addresses of already contiguous microtiles; it performs no gather. PREPARE_KV stores K digits as 16-key $\times$ 128-feature blocks and V digits as 16-feature $\times$ 128-key blocks, grouped by the 256-key tile. SPLIT_AND_PACK_LEFT creates the matching left operands.

Scheduling and performance. SA32 owns rows 0-95; SA16 owns rows 96-127. This division is valid but needs timing-based tuning. Every physical launch costs 100 communication cycles; each result drains through the array's 8 B/cycle output port. Digit splitting adds four products per reduction segment. Include those launches, packing, and reconstruction in performance estimates. The assignment's unbounded vector-compute assumption also warrants comparing a vector-only product.

Public algorithm reference: Tri Dao, FlashAttention-2, Algorithm 1. Hardware scheduling and overflow handling here are adaptations to the supplied architecture.